

Atomspace Interaction

add-reduct

Description: Reduces an atom and adds it to the atomspace.

Parameters:

- Space: The atomspace to add the atom to.
- Atom: The atom to be added.

Return: Unit atom

Example:

```
!(add-reduct &self (= (add) (+ 1 3))); This will add (= (add) 4) to the working space, &self
```

add-atom

Description: Adds an atom into the atomspace without reducing it.

Parameters:

- Space: Atomspace to add atom into
- Atom: Atom to be added

Return: Unit atom

Example:

```
!(add-atom &self (= (add) (+ 1 3))); This will add (= (add) (+ 1 3)) to the working space, &self
```

get-type

  latest ▼

Description: Returns type notation of input atom.

Parameters:

- Atom: Atom to get type for

Return: Type notation or %Undefined% if there is no type for input Atom

Example:

```
!(get-type 1) ; Returns Number
```

get-type-space

Description: Returns type notation of input Atom relative to a specified atomspace.

Parameters:

- Space: Atomspace where type notation for input atom will be searched
- Atom: Atom to get type for

Return: Type notation or %Undefined% if there is no type for input Atom in provided atomspace

Example:

```
(: a A)  
!(get-type-space &self a); Returns A because we defined the type of a to be A in the working  
space, &self
```

get-metatype

Description: Returns metatype of the input atom

Parameters:

- Atom: Atom to get metatype for

Return: Metatype of input atom

Example:

```
!(get-metatype True); Returns Grounded  
!(get-metatype (a b)); Return Expression
```

if-equal

Description: Checks if first two arguments are equal and evaluates third argument if equal, fourth argument - otherwise.

Parameters:

- Arg1: First argument
- Arg2: Second argument
- Then: Atom to be evaluated if arguments are equal
- Else: Atom to be evaluated if arguments are not equal

Return: Evaluated third or fourth argument

Example:

```
!(if-equal 1 1 "Equal" "Not Equal"); Returns "Equal"  
!(if-equal 1 2 "Equal" "Not Equal"); Returns "Not Equal"
```

new-space

Description: Creates new Atomspace which could be used further in the program as a separate from &self Atomspace

Parameters:

- None

Return: Reference to a new space

Example:

```
!(new-space); Returns reference to the new space
```

new-state

Description: Creates a new state atom wrapping its argument

Parameters:

- Atom: The atom to be wrapped

Return: (State \$value) where \$value is an argument to the new-state

Example:

```
!(new-state rest); Returns (State rest)
```

change-state!

Description: Changes the input state's wrapped atom to another atom (the second argument)

Parameters:

- StateAtom: State to be changed
- Atom: The new value for the state to replace the wrapped atom

Return: (State \$value) where \$value is the new atom

Example:

```
!(bind! state (new-state rest)) !(change-state! state active); Returns (State active)
```

get-state

Description: Used to get the wrapped atom in the given state

Parameters:

- StateAtom: The state

Return: Atom which is wrapped by the given state

Example:

```
!(bind! state (new-state rest)) !(get-state state); Returns rest
```

remove-atom

Description: Removes atom from the input Atomspace

Parameters:

- Space: Reference to the space from which the Atom needs to be removed
- Atom: Atom to be removed

Return: Unit atom

Example:

```
!(remove-atom &self (= (add) 4)); Removes (= (add) 4) from the working space, &self
```

get-atoms

Description: Shows all atoms in the input Atomspace

Parameters:

- Space: Reference to the space

Return: List of all atoms in the input space

Example:

```
!(get-atoms &self); Returns all atoms inside &self
```

match

Description: Searches for all declared atoms corresponding to the given pattern inside space and returns the output template

Parameters:

- Space: Atomspace to search pattern
- Pattern: Pattern atom to be searched
- Output: Output template typically containing variables from the input pattern

Return: If match was successfull it outputs template with filled variables using matched pattern.
Empty - otherwise

Example:

```
(= (add) (+ 1 2))  
(= (add) (+ 4 2))  
!(match &self (= (add) (+ $x $y)) $x); Returns 1, 4
```